

Grafos

Prof. Dr. Matheus da Silva Menezes



Unidade 03

GRAFOS

GRAFOS
Heurísticas para o PCV

Cronograma



1 Unidade

Introdução e Buscas

1. **Conceitos Básicos**
 - a. Definições e Classificações
 - b. Estruturas de Dados
2. **Buscas**
 - a. Busca em Largura
 - b. Busca em Profundidade
3. **Biconectividade**
4. **Buscas em Grafos Direcionados**

2 Unidade

Árvores, Caminhos e Fluxos

1. **Árvores**
 - a. Kruskal
 - b. Prim
 - c. Boruvka
 - d. Chu-Liu
2. **Caminho mais curto**
 - a. Dijkstra
 - b. Bellman-Ford
 - c. Floyd-Warshall
3. **Ciclo Euleriano**

3 Unidade

Planaridade e Prob. NP-Difíceis

1. **Fluxo em Redes**
2. **Emparelhamento**
3. **Planaridade**
4. **Ciclo Hamiltoniano e PCV**
5. **Coloração**
6. **Conjuntos de Vértices e Arestas**

14/04/26 Avaliação 01

26/05/26 Avaliação 02

07/07/26 Avaliação 03

09/07/26 Final

MARÇO						
D	S	T	Q	Q	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

ABRIL						
D	S	T	Q	Q	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

MAIO						
D	S	T	Q	Q	S	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

JUNHO						
D	S	T	Q	Q	S	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

JULHO						
D	S	T	Q	Q	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

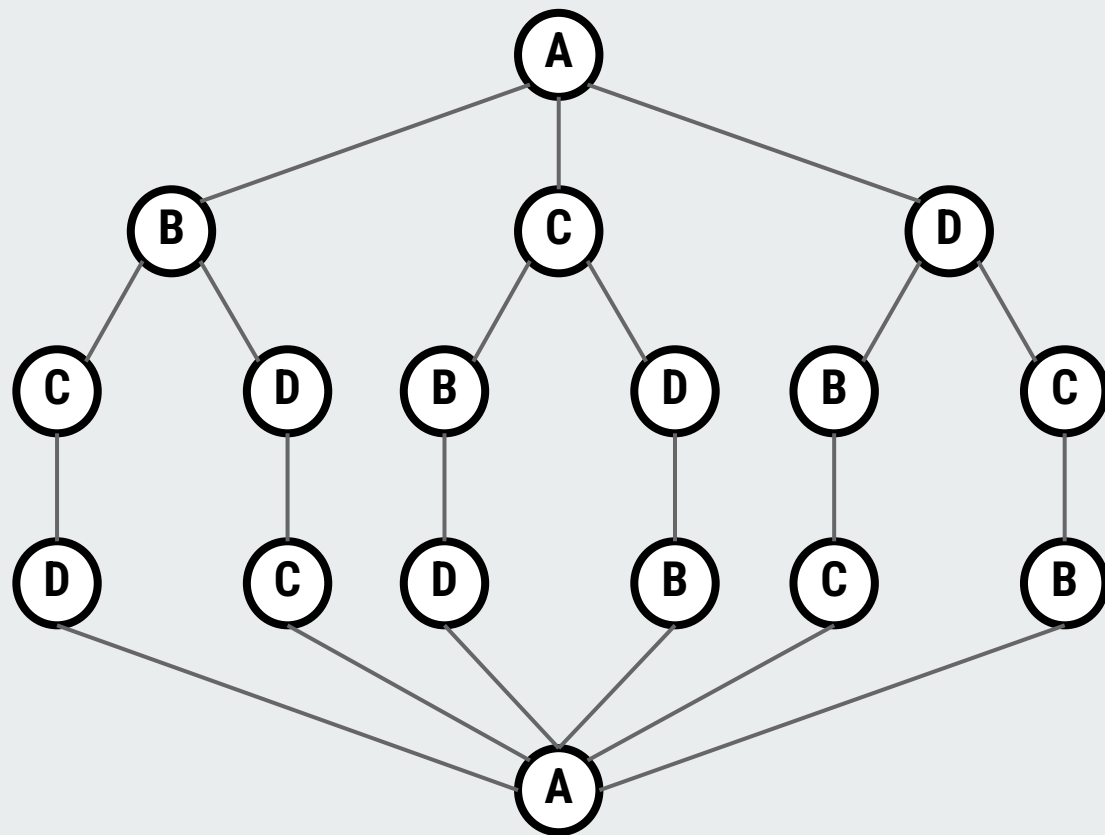
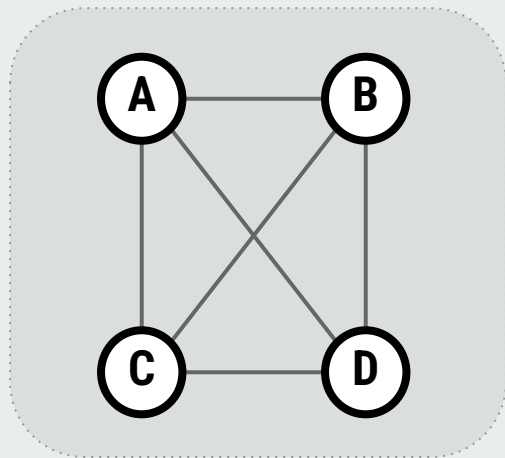
Definição

O Problema do Caixeiro Viajante (Traveling Salesman Problem)

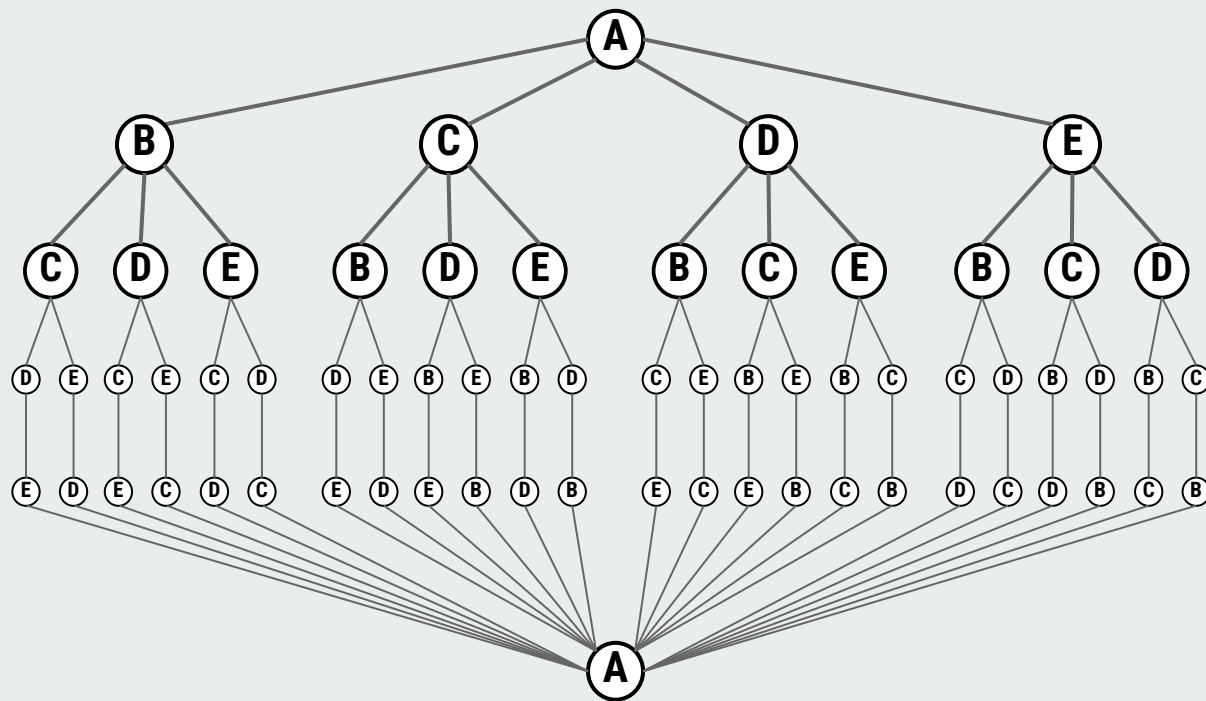
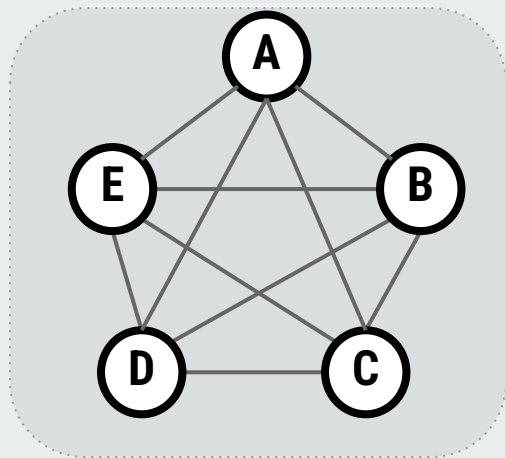
*Dado um grafo ponderado $G = (V, A)$, encontrar um **ciclo** H cuja soma das arestas seja mínima dentre todos os ciclos geradores de G .*

"Dado N cidades, achar a caminho mais curto passando por todas as cidades uma única vez."





Para N cidades há $(N-1)!$ Rotas
 $(4-1)! = 3! = \mathbf{6 \text{ rotas}}$



Para N cidades há $(N-1)!$ Rotas
 $(5-1)! = 4! = \mathbf{24 \text{ rotas}}$

Para **N** cidades há **$(N-1)!$** Rotas.

N	# Rotas
7	720
8	5.040
9	40.320
10	362.880
11	3.628.800
12	39.916.800
26	15.511.210.043.330.985.984.000.000

Função tempo/ complexidade	Quantidade de Dados: N				
	10	20	30	40	50
N	0,00001s	0,00002s	0,00003s	0,00004s	0,00005s
N^2	0,0001s	0,0004s	0,0009s	0,0016s	0,0036s
N^3	0,001s	0,008s	0,027s	0,064s	0,125s
2^N	0,001s	1,0s	17,19 min	12,7 dias	35,7 anos
3^N	0,059s	58 min	6,5 anos	3.855 séculos	200.000.000 séculos

Obs: usando um computador capaz de realizar um milhão de operações aritméticas básicas por segundo.

Um problema **simétrico** com n cidades possui $(n-1)!/2$ soluções.

Se $n=20$, então existem mais de 10^{18} soluções.

Karp (1975) provou que o problema pertence à classe NP-difícil.

O número de segundos desde o Big Bang até hoje é de aproximadamente $4,35 \times 10^{17}$ segundos

Duas classes de algoritmos são usados para resolver problemas de otimização combinatória:

- *Algoritmos exatos.*
- *Técnicas heurísticas (ou algoritmos aproximados).*
 - **Heurísticas construtivas:** buscam construir uma solução viável a partir do zero, escolhendo vértices ou arestas progressivamente, com base em algum critério local

Heurística do vizinho mais próximo (*Nearest Neighbor*)

- *O algoritmo do vizinho mais próximo é guloso (greedy):*
- *Começa de um vértice inicial **s**;*
- *Em cada passo, escolhe o vértice não visitado mais próximo;*
- *Repete até visitar todos os vértices;*
- *Retorna ao vértice inicial, formando um ciclo hamiltoniano;*
- *Não garante a solução ótima;*

Na aula passada...

Heurística de Inserção Mais Próxima (*Nearest Insertion* – NI)

- *Começa com um pequeno ciclo (por exemplo, o vértice inicial e o mais próximo dele).*
- *Depois, insere iterativamente o vértice mais próximo de qualquer vértice do ciclo atual, na posição que minimiza o aumento no custo total*

Insercao_mais_proxima(Grafo G , vértice s , matriz de distâncias d)

v_0 = vértice mais próximo de s // escolha gulosa

$rota \leftarrow [s, v_0, s]$

$C \leftarrow \{s, v_0\}$ // vértices já inseridos

$R \leftarrow V \setminus C$ // vértices restantes

Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \text{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

$melhor_custo \leftarrow custo$

$melhor_pos \leftarrow i$

Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

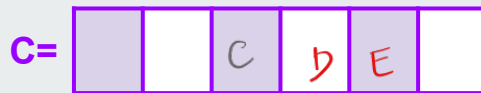
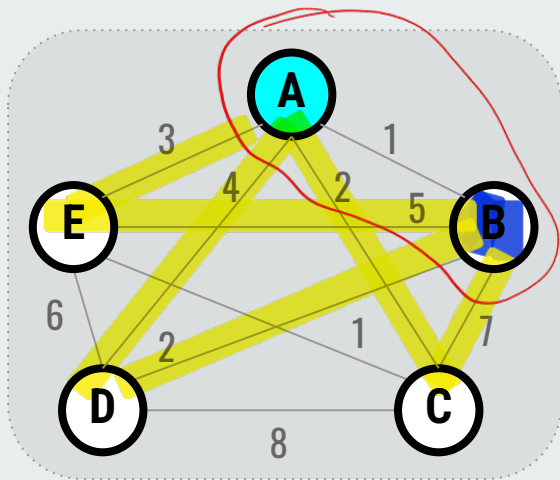
Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R

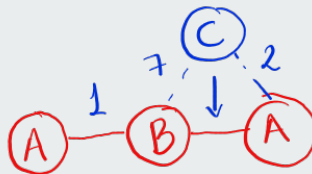
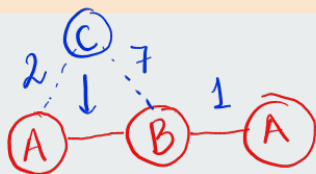
Retorne $(rota)$

C
↑

R
↑



rota = {A, B}



Insercao_mais_proxima(Grafo G, s, d)

v_0 = vértice mais próximo de s // escolha gulosa

$rota \leftarrow [s, v_0, s]$

$C \leftarrow \{s, v_0\}$ // vértices já inseridos

$R \leftarrow V \setminus C$ // vértices restantes

Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \text{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

$melhor_custo \leftarrow custo$

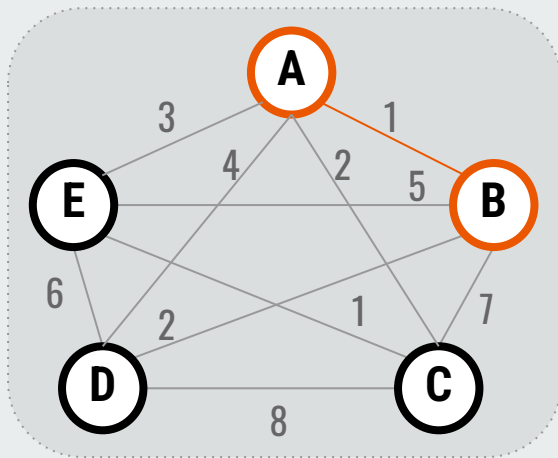
$melhor_pos \leftarrow i$

Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R

Retorne ($rota$)



$C =$

A	B				
---	---	--	--	--	--

$R =$

C	D	E			
---	---	---	--	--	--

Insercao_mais_proxima(Grafo G , s , d)

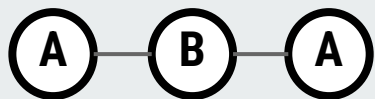
v_0 = vértice mais próximo de s // escolha gulosa

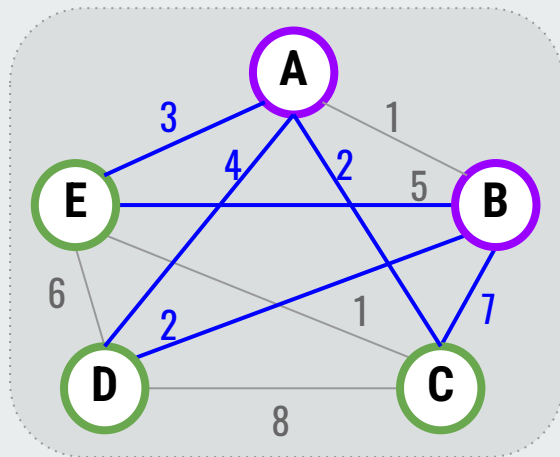
$rota \leftarrow [s, v_0, s]$

$C \leftarrow \{s, v_0\}$ // vértices já inseridos

$R \leftarrow V \setminus C$ // vértices restantes

$rota = \{A, B, A\}$





rota = { A , B , A }

C =

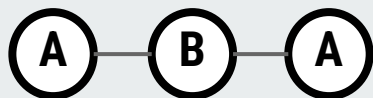
A	B				
---	---	--	--	--	--

R =

C	D	E			
---	---	---	--	--	--

$r^* = C$

melhor_custo =
melhor_pos =



Enquanto $R \neq \emptyset$ faça

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

dist $r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \operatorname{argmin}_r(\text{dist}_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

melhor_custo $\leftarrow \infty$, **melhor_pos** $\leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

u $\leftarrow \text{rota}[i]$

v $\leftarrow \text{rota}[i+1]$

custo $\leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se **custo** $<$ **melhor_custo** **então**

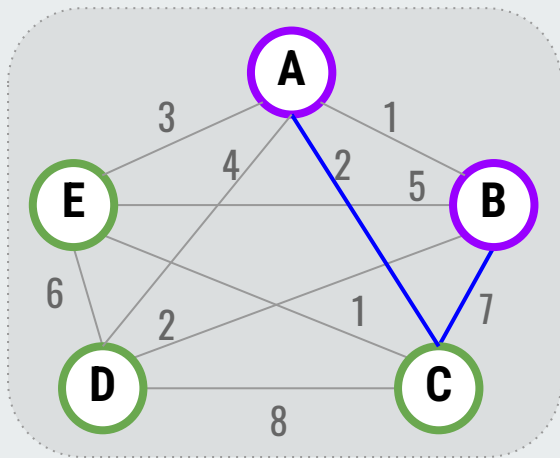
melhor_custo $\leftarrow \text{custo}$

melhor_pos $\leftarrow i$

Inserir r^* entre $\text{rota}[\text{melhor_pos}]$ e $\text{rota}[\text{melhor_pos}+1]$

Adicionar r^* em **C** // atualiza o conjunto C

Remover r^* de **R** // atualiza o conjunto R



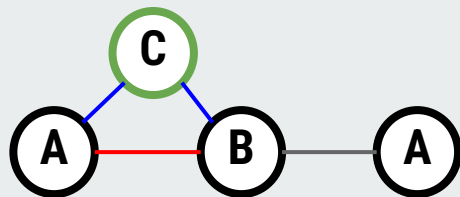
rota = { A, C, B, A }

C = [A, B, C, , ,]

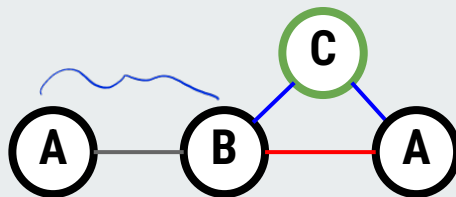
R = [C, D, E, , ,]

$r^* = C$

melhor_custo = 8
melhor_pos = 1



$$\begin{aligned} & d(u, r^*) + d(r^*, v) - d(u, v) \\ & d(A, C) + d(C, B) - d(A, B) \\ & \text{custo} = 2 + 7 - 1 = 8 \end{aligned}$$



$$\begin{aligned} & d(u, r^*) + d(r^*, v) - d(u, v) \\ & d(B, C) + d(C, A) - d(B, A) \\ & \text{custo} = 7 + 2 - 1 = 8 \end{aligned}$$

Enquanto $R \neq \emptyset$ faça

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ faça

$\text{dist}_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \text{argmin}_r(\text{dist}_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$\text{melhor_custo} \leftarrow \infty$, $\text{melhor_pos} \leftarrow \text{NULL}$

Para i de 1 até $|rota|-1$ faça

$u \leftarrow \text{rota}[i]$

$v \leftarrow \text{rota}[i+1]$

$\text{custo} \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $\text{custo} < \text{melhor_custo}$ então

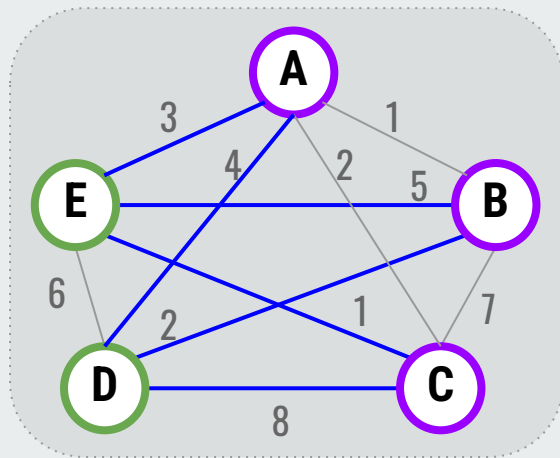
$\text{melhor_custo} \leftarrow \text{custo}$

$\text{melhor_pos} \leftarrow i$

Inserir r^* entre $\text{rota}[\text{melhor_pos}]$ e $\text{rota}[\text{melhor_pos}+1]$

Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R



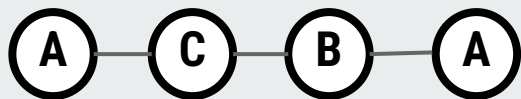
rota = { A , C , B , A }

C = [A | B | C | | |]

R = [D | E | | | |]

$r^* = E$

melhor_custo =
melhor_pos =



Enquanto $R \neq \emptyset$ faça

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ faça

$\text{dist}_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \text{argmin}_r(\text{dist}_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$\text{melhor_custo} \leftarrow \infty$, $\text{melhor_pos} \leftarrow \text{NULL}$

Para i de 1 até $|rota|-1$ faça

$u \leftarrow \text{rota}[i]$

$v \leftarrow \text{rota}[i+1]$

$\text{custo} \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $\text{custo} < \text{melhor_custo}$ então

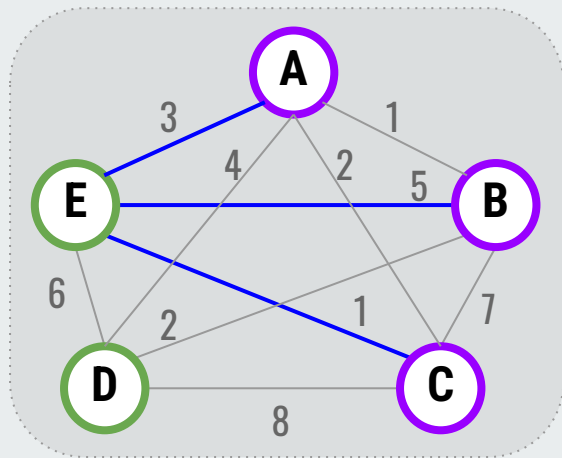
$\text{melhor_custo} \leftarrow \text{custo}$

$\text{melhor_pos} \leftarrow i$

Inserir r^* entre $\text{rota}[\text{melhor_pos}]$ e $\text{rota}[\text{melhor_pos}+1]$

Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R



$rota = \{A, C, E, B, A\}$

$C =$

A	B	C	E		
---	---	---	---	--	--

$R =$

D	E				
---	---	--	--	--	--

$r^* = E$

$melhor_custo = -1$
 $melhor_pos = 2$

Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \operatorname{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

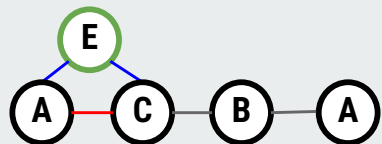
$melhor_custo \leftarrow custo$

$melhor_pos \leftarrow i$

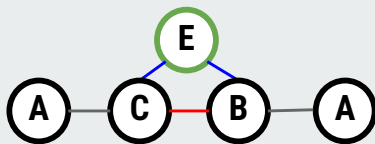
Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

Adicionar r^* em C // atualiza o conjunto C

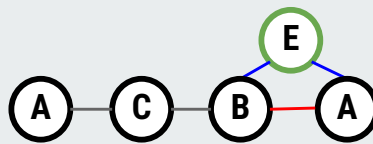
Remover r^* de R // atualiza o conjunto R



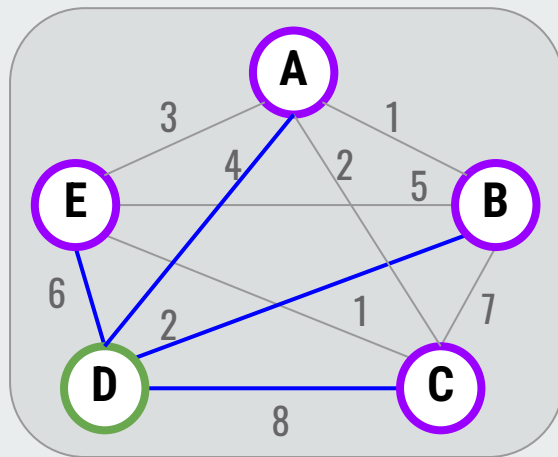
$d(A, E) + d(E, C) - d(A, C)$
 $custo = 3 + 1 - 2 = 2$



$d(C, E) + d(E, B) - d(C, B)$
 $custo = 1 + 5 - 7 = -1$



$d(B, E) + d(E, A) - d(B, A)$
 $custo = 5 + 3 - 1 = 7$



$rota = \{A, C, E, B, A\}$

$C =$

A	B	C	E		
---	---	---	---	--	--

$R =$

D					
---	--	--	--	--	--

$r^* = D$

$melhor_custo =$
 $melhor_pos =$



Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \operatorname{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

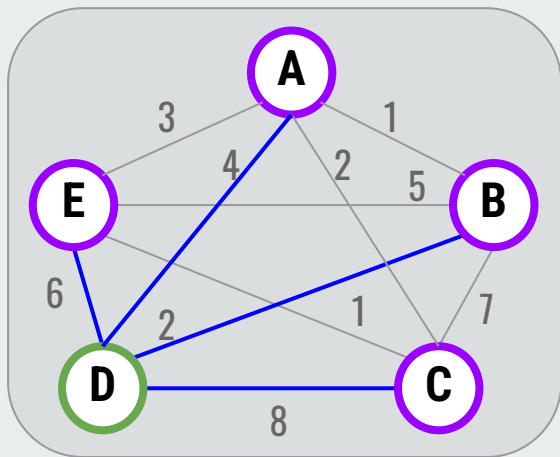
$melhor_custo \leftarrow custo$

$melhor_pos \leftarrow i$

Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R



$rota = \{A, C, E, D, B, A\}$

$C =$

A	B	C	E	D	
---	---	---	---	---	--

$R =$

D					
---	--	--	--	--	--

$r^* = D$

$melhor_custo = 3$
 $melhor_pos = 3$

Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \operatorname{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

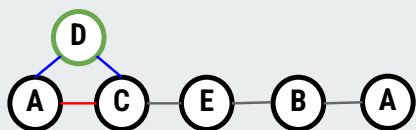
$melhor_custo \leftarrow custo$

$melhor_pos \leftarrow i$

Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

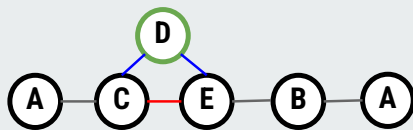
Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R



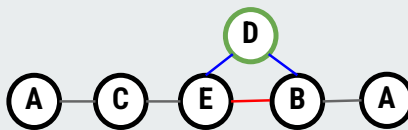
$$d(A, D) + d(D, C) - d(A, C)$$

$$custo = 4 + 8 - 2 = 10$$



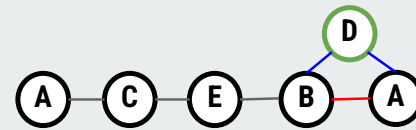
$$d(C, D) + d(D, E) - d(C, E)$$

$$custo = 8 + 6 - 1 = 13$$



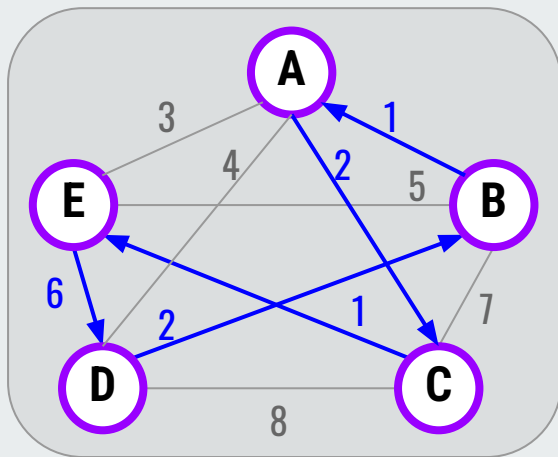
$$d(E, D) + d(D, B) - d(E, B)$$

$$custo = 6 + 2 - 5 = 3$$



$$d(B, D) + d(D, A) - d(B, A)$$

$$custo = 2 + 4 - 1 = 5$$



$C =$

A	B	C	E	D	
---	---	---	---	---	--

$R =$

--	--	--	--	--	--

$rota = \{ A, C, E, D, B, A \}$



$custo = 2 + 1 + 6 + 2 + 1 = 12$

Insercao_mais_proxima(Grafo G, s, d)

v_0 = vértice mais próximo de s // escolha gulosa

$rota \leftarrow [s, v_0, s]$

$C \leftarrow \{s, v_0\}$ // vértices já inseridos

$R \leftarrow V \setminus C$ // vértices restantes

Enquanto $R \neq \emptyset$ **faça**

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice $r \in R$ **faça**

$dist_r \leftarrow \min\{d(r, c) \mid c \in C\}$

Escolha $r^* = \text{argmin}_r(dist_r)$

// Passo 2: encontrar melhor posição para inserção de r^*

$melhor_custo \leftarrow \infty$, $melhor_pos \leftarrow \text{NULL}$

Para i **de** 1 **até** $|rota|-1$ **faça**

$u \leftarrow rota[i]$

$v \leftarrow rota[i+1]$

$custo \leftarrow d(u, r^*) + d(r^*, v) - d(u, v)$

Se $custo < melhor_custo$ **então**

$melhor_custo \leftarrow custo$

$melhor_pos \leftarrow i$

Inserir r^* entre $rota[melhor_pos]$ e $rota[melhor_pos+1]$

Adicionar r^* em C // atualiza o conjunto C

Remover r^* de R // atualiza o conjunto R

Retorne ($rota$)

COMPLEXIDADE?

Insercao_mais_proxima(Grafo **G**, vértice **s**, matriz de distâncias **d**)

v₀ = vértice mais próximo de **s** // escolha gulosa

rota ← [**s**, **v**₀, **s**]

C ← {**s**, **v**₀} // vértices já inseridos

R ← **V** \ **C** // vértices restantes

Enquanto **R** ≠ ∅ faça

// Passo 1: escolher o vértice mais próximo do ciclo atual

Para cada vértice **r** ∈ **R** faça

dist_r ← min{ **d**(**r**, **c**) | **c** ∈ **C** }

Escolha **r*** = argmin_r(**dist_r**)

// Passo 2: encontrar melhor posição para inserção de **r***

melhor_custo ← ∞, **melhor_pos** ← NULL

Para **i** de 1 até |**rota**|-1 faça

u ← **rota**[**i**]

v ← **rota**[**i**+1]

custo ← **d**(**u**, **r***) + **d**(**r***, **v**) - **d**(**u**, **v**)

Se **custo** < **melhor_custo** então

melhor_custo ← **custo**

melhor_pos ← **i**

Inserir **r*** entre **rota**[**melhor_pos**] e **rota**[**melhor_pos**+1]

Adicionar **r*** em **C** // atualiza o conjunto C

Remover **r*** de **R** // atualiza o conjunto R

Retorne (**rota**)

$\Theta(n^2)$

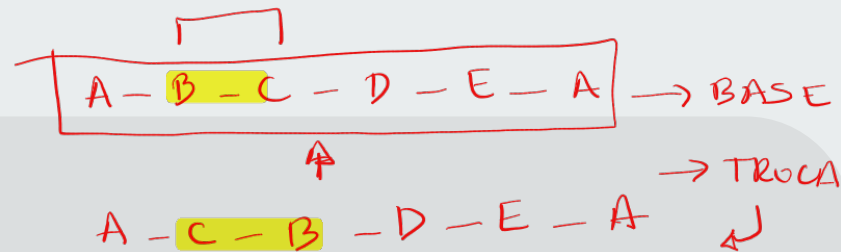
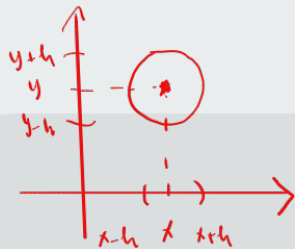
Outras Heurísticas Construtivas

Heurística do Caixeiro Mais Barato (Cheapest Insertion – CI): Em cada iteração, avalia todos os vértices não visitados e todas as posições possíveis para inserção; escolhe a combinação que propicia o menor aumento do custo total.

Heurística da Árvore Geradora Mínima (MST Heuristic): Constrói uma árvore geradora mínima (MST). Faz uma busca em profundidade (DFS) nessa árvore, retornando ao início. Remove vértices repetidos para formar o ciclo.

Inserção Aleatória (Random Insertion – RI): Escolhe vértices aleatoriamente e insere-os na melhor posição possível da rota atual.

...

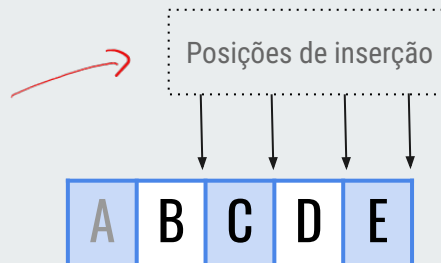
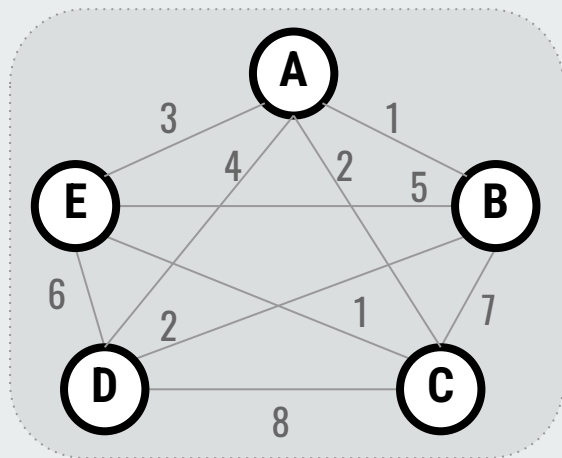


Heurísticas de Melhoramento Local

- *Partem de uma solução existente e tentam melhorá-la explorando sua vizinhança.*
- *Também conhecidas como buscas locais.*

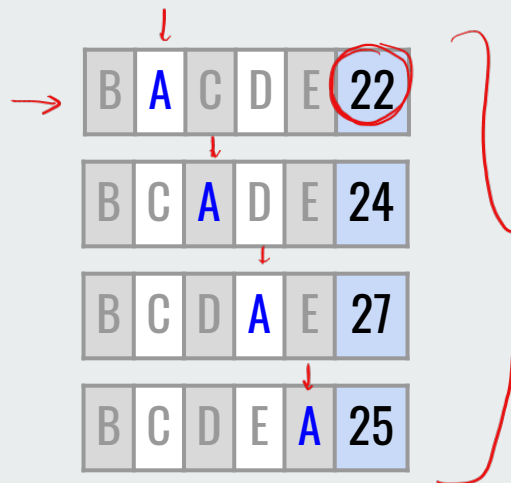
Algumas Heurísticas de Melhoria Local

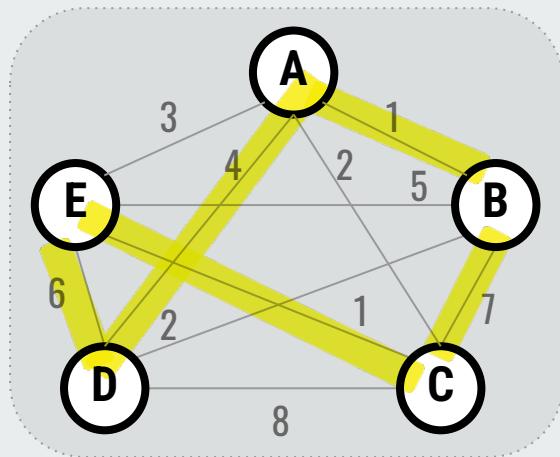
- **Shift**: Remove uma cidade de uma posição e reinsere em outra posição da rota.
- **Swap**: Troca a posição de dois vértices (ou cidades) na sequência da rota.
- **Inversão**: Seleciona dois índices $i < j$ e inverte a ordem dos vértices entre eles.
- **2-OPT**: remove 2 arestas e reconecta de outra forma
- ...



Shift: Remove uma cidade de uma posição e reinsere em outra posição da rota.

Sol= **A** B C D E $1 + 7 + 8 + 6 + 3 = 25$





Troca de posição
com



Swap: Troca a posição de dois vértices (ou cidades) na sequência da rota.

Sol=

A	B	C	D	E
---	---	---	---	---

 $\rightarrow A \ 1 + 7 + 8 + 6 + 3 = 25$

D *A E-D*

B	A	C	D	E	22
---	---	---	---	---	----

$\rightarrow$

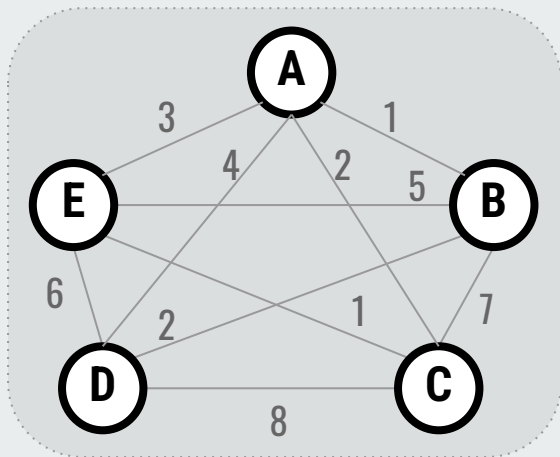
C	B	A	D	E	19
---	---	---	---	---	----

D	B	C	A	E	20
---	---	---	---	---	----

E	B	C	D	A	27
---	---	---	---	---	----

BACDE

SHIFT
SWAP
INV

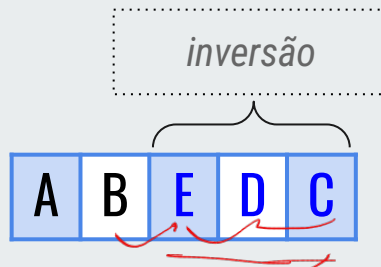
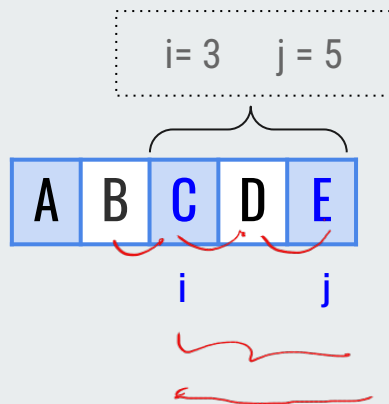


Inversão: Seleciona dois índices $i < j$ e inverte a ordem dos vértices entre eles.

Sol=

A	B	C	D	E
---	---	---	---	---

$$1 + 7 + 8 + 6 + 3 = 25$$



$$1 + 5 + 6 + 8 + 2 = 22$$

Até quando executar a busca local?

- **First Improvement:** *Até encontrar a primeira melhoria (e reinicia o processo)*
- **Best Improvement:** *Após explorar toda a vizinhança (e reinicia a partir da melhor solução)*

Busca_Local(solução **S**, tipo_busca **TipoBusca**)

— **Melhoria** = **V**

Enquanto (**Melhoria**) **faça**

— **Melhoria** = **F**

Se (**TipoBusca** = "First_Imp") **então**

Para cada **S'** $\in$ **Vizinhança**(**S**) **faça**

Se (**Custo**(**S'**) < **Custo**(**S**)) **então**

S = **S'**,

 — **Melhoria** = **V**

 — **Pare;** // reinicia a busca a partir desta solução ←

Senão se (**TipoBusca** = "Best_Imp") **então**

S_best = **S**

Para cada **S'** $\in$ **Vizinhança**(**S**) **faça**

Se (**Custo**(**S'**) < **Custo**(**S_best**)) **então**

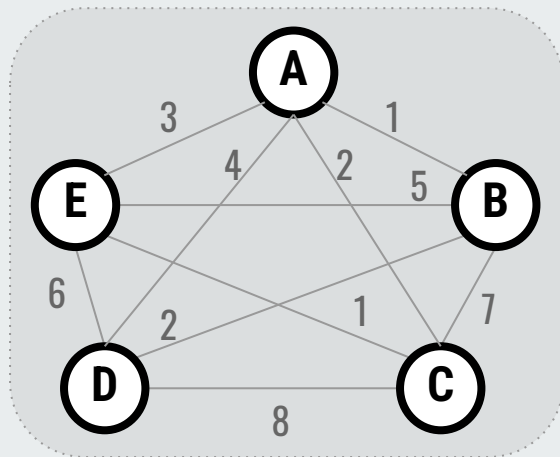
S_best = **S'** ←

Se (**Custo**(**S_best**) < **Custo**(**S**)) **então**

S = **S_best** ←

Melhoria = **V**

Retorne (**S**)



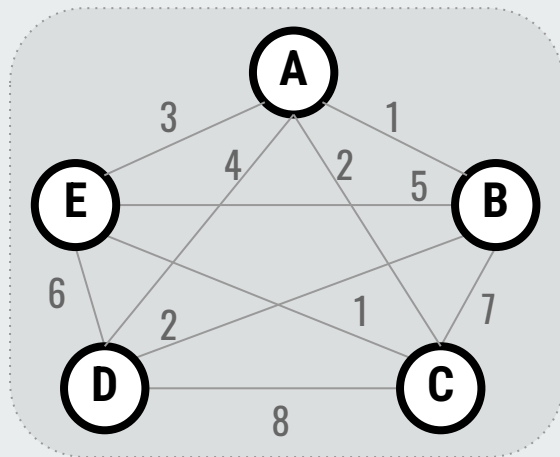
Shift: Remove uma cidade de uma posição e reinsere em outra posição da rota.

Sol=

A	B	C	D	E
---	---	---	---	---

$$1 + 7 + 8 + 6 + 3 = 25$$

B A C D E 22	B A C D E 22	C A B D E 14	D A B C E 19	E A B C D 25
B C A D E 24	A C B D E 20	A C B D E 20	A D B C E 17	A E B C D 27
B C D A E 27	A C D B E 20	A B D C E 15	A B D C E 15	A B E C D 19
B C D E A 25	A C D E B 22	A B D E C 12	A B C E D 19	A B C E D 19



Swap: Troca a posição de dois vértices (ou cidades) na sequência da rota.

Sol=

A	B	C	D	E
---	---	---	---	---

$$1 + 7 + 8 + 6 + 3 = 25$$

B	A	C	D	E	22
C	B	A	D	E	19
D	B	C	A	E	20
E	B	C	D	A	27
A	C	B	D	E	20
A	D	C	B	E	27
A	C	B	D	E	20
A	D	C	B	E	27
A	B	D	C	E	15
A	B	D	C	E	15
A	B	E	D	C	22
A	B	C	E	D	19
A	E	C	D	B	15
A	B	C	E	D	19

Algoritmo Heurístico Composto

Heurística Construtiva

Obtém uma solução inicial para o problema. No caso do PCV, uma rota.



Busca Local

Tenta melhorar a solução obtida na fase anterior



Uma Melhor Solução

A combinação de duas estratégias amplia as chances de se conseguir uma boa solução para o problema.

Vamos programar?



1. Implemente o Método da **inserção mais próxima**, de forma a obter um percurso (ciclo) e o custo deste ciclo, a partir de um vértice inicial **s**.
2. Implemente as buscas locais **Shift**, **Swap** e **Inversão**.
3. Implemente um **algoritmo heurístico composto**, combinando *Heurísticas Construtivas* e *Buscas Locais*.



(c) Shane Keena / www.comedywildlifephotography.com

Dúvidas?!

